Fast LIO 2

By: > *terminal_gremlin (c-lydia^^)*
Chheng Lydia

Source

- Authors / lecturer: Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, Fu Zhang
- Year: 2021 (arXiv preprint)
- Paper: FAST-LIO2: Fast Direct LiDAR-inertial Odometry
- DOI: 10.48550/arXiv.2107.06829
- Full paper · PDF
- Implementation used below: hku-mars/FAST_LIO, ROS2 branch pinned to `cc3c9e8b1469b33b8072764d6db2348ed6dc597f`.

Question

How does Fast-LIO2 achieve real-time, highly accurate 3D LiDAR-Inertial state estimation and mapping without requiring hand-engineered feature extraction?

Core idea

Fast-LIO2 is a direct, tightly-coupled LiDAR–Inertial Odometry framework that registers raw LiDAR points directly to a global or local map without extracting edge or planar features. It maintains large, dense point cloud maps using a dynamic Incremental KD-Tree (`ikd-Tree`) that supports point insertion, on-tree downsampling, lazy box-wise deletions, and parallel subtree re-balancing. State estimation is formulated on manifold spaces using a tightly-coupled iterated Extended Kalman Filter (iEKF) with a state-dimension Kalman gain computation that supports the authors' reported odometry and mapping rates up to 100 Hz in their experiments. This simulation uses 10 Hz scans; it does not inherit a 100 Hz performance guarantee.

Method and assumptions

Method

1. **IMU Propagation and Motion Compensation (De-skewing):** propagates the state forward using high-rate IMU measurements, then compensates scan points backward relative to the scan end-time using their individual sampling timestamps. This reduces motion distortion; it does not guarantee exact correction when measurements, calibration, or timing are imperfect.

2. **Direct raw Point Scan-to-Map Matching:** registers raw 3D points directly to the map via point-to-plane residual distances, removing hand-engineered feature extraction and adapting naturally to diverse spinning and solid-state scan patterns
3. **On-Manifold IEKF State Update:** propagates and updates system states directly on the manifold space $\mathcal{M} = SO(3) \times \mathbb{R}^{15} \times SO(3) \times \mathbb{R}^3$
4. **Dynamic Map Management (`ikd-Tree`):** organizes map points in a local cuboid with side length L (the paper's default evaluation uses $L = 1000\,\mathrm{m}$). Deletes historical points falling outside the active map region via box-wise delete operations and inserts new points with on-tree spatial downsampling. Sub-tree re-balancing is offloaded to a parallel second thread to reduce blocking of the main thread

Assumptions

- **Rigid Sensor Extrinsics and Timing:** the LiDAR and IMU are rigidly mounted. Their relative transform must be known or estimated, and point/IMU timestamps must have a consistent time base. The paper includes extrinsic estimation in the state; this simulation uses known fixed extrinsics and disables their online estimation. Simulation timestamps are valid inputs; hardware timestamps are not mandatory.
- **Local Environmental Planarity:** neighboring map points locally approximate planar patches to support point-to-plane residuals. The scene also needs sufficient geometric constraints to observe motion; a dominant flat surface can leave some directions poorly constrained.

Equations and notation

Defined Symbols and Units

- G: global reference frame (coinciding with the initial IMU frame)
- I: IMU body frame
- L: local LiDAR sensor frame
- $T_I^G = (R_I^G, p_I^G) \in SE(3)$: IMU rotation matrix ($SO(3)$) and position vector ($\mathbb{R}^3$, meters) in the global frame
- $T_L^I = (R_L^I, p_L^I) \in SE(3)$: LiDAR-to-IMU extrinsic rotation matrix and translation vector
- $\mathbf{x} \in \mathcal{M}$: state on the paper's 24-dimensional manifold

$$\mathcal{M} = SO(3) \times \mathbb{R}^{15} \times SO(3) \times \mathbb{R}^3 : \mathbf{x} = \left[R_I^G, p_I^G, v_I^G, b_\omega, b_a, g^G, R_L^I, p_L^I\right]^T$$

- $\mathbf{u} = \left[\omega_m^T, a_m^T\right]^T$: measured angular velocity (rad/s) and linear acceleration (m/s^2)
- $\mathbf{w} = [n_\omega^T, n_a^T, n_{b\omega}^T, n_{ba}^T]^T$: sensor measurement noise and bias random walk noise processes
- ${}^L p_j \in \mathbb{R}^3$: j-th measured 3D point in the local LiDAR frame L (meters)

- ${}^{G}u_j \in \mathbb{R}^3$: unit normal vector of the corresponding local planar patch in global frame
- ${}^{G}q_j \in \mathbb{R}^3$: centroid point lying on the local planar patch in global frame

The paper uses the 24-dimensional manifold above. The pinned implementation represents fixed-magnitude gravity on S^2, giving a 23-dimensional error state. This is an implementation distinction, not a correction to the paper's stated dimension.

The kinematic equations use acceleration in $\mathrm{m/s^2}$. The simulated Livox topic follows this driver's convention and publishes acceleration in **g**; FAST-LIO rescales it during IMU processing.

Formulations

Continuous Kinematic State Transition model

$$\dot{R}_I^G = R_I^G \lfloor \omega_m - b_\omega - n_\omega \rfloor_\wedge, \quad \dot{p}_I^G = v_I^G, \quad \dot{v}_I^G = R_I^G (a_m - b_a - n_a) + g^G$$

Implicit Point-to-Plane Measurement Residual (z_j)

$$z_j = h_j(\hat{x}_k, 0) = {}^{G}u_j^T \left(\hat{R}_I^G \hat{R}_L^I {}^{L}p_j + \hat{R}_I^G \hat{p}_L^I + \hat{p}_I^G - {}^{G}q_j \right)$$

The subtraction of the plane point q_j is **inside** the inner product, so z_j is a scalar signed distance in meters. The residual at an estimated state is generally nonzero. The ideal noise-free constraint is zero; the iterated filter reduces the residual while accounting for the prior and measurement uncertainty. Paper, §IV-B2, Eq. (9)

State-Dimension Computationally Efficient Kalman Gain (K)

$$K = (H^T R^{-1} H + P^{-1})^{-1} H^T R^{-1}$$

> Here H is the stacked measurement Jacobian, R the measurement-noise covariance, and P the prior covariance expressed in the current error coordinates. The formula inverts a state-sized matrix instead of a measurement-sized matrix. For the manifold iteration, the prior covariance is transported into the current tangent coordinates; see the paper's Eqs. (11)–(14).

`ikd-Tree` Balancing Criteria

- **α-Balanced Criterion:** $S(T.\text{leftchild}) < \alpha_{\text{bal}} S(T) - 1$ and $S(T.\text{rightchild}) < \alpha_{\text{bal}} S(T) - 1$ (where $\alpha_{\text{bal}} \in (0.5, 1)$)
- **α-Deleted Criterion:** $I(T) < \alpha_{\text{del}} S(T)$ (where $I(T)$ is the invalid/deleted node count and $\alpha_{\text{del}} \in (0, 1)$). Violating either criterion triggers rebalancing; these inequalities describe a balanced tree.

Evidence and limitations

Author Claims and Benchmark Evidence

These are the paper's experimental results, not guarantees or measurements of this robot simulation.

- **Real-time performance:** reported operation up to 100 Hz on Intel i7-based hardware and 10 Hz on the Khadas VIM3 ARM platform under the evaluated workloads. §VI-D and §VII
- **Data structure efficiency:** the reported ikd-Tree experiments show lower overall update/search cost than the compared octree, R^*-tree and nanoflann structures. Peak processing time was 214.4 ms in `nclt_2` and below 150 ms in the other 17 evaluated data-structure sequences; nanoflann exceeded seconds in the long sequences. These are data-structure evaluation timings, not a blanket end-to-end latency bound. §VI-B2
- **Benchmark comparison:** 19 sequences from `lili`, `utbm`, `ulhk`, `nclt` and `liosam`. The authors report approximately $8\times$, $10\times$ and $6\times$ faster combined odometry/mapping processing than LILI-OM, LIO-SAM and LINS, respectively, with favorable accuracy in their evaluation. This does not mean it wins every individual table entry. §VI-C and §VI-D
- **Aggressive motion:** experiments include handheld motion reaching $7\,\mathrm{m/s}$ and a UAV flip reaching $1198^\circ/\mathrm{s}$. §VII-B2 and §VII-B3

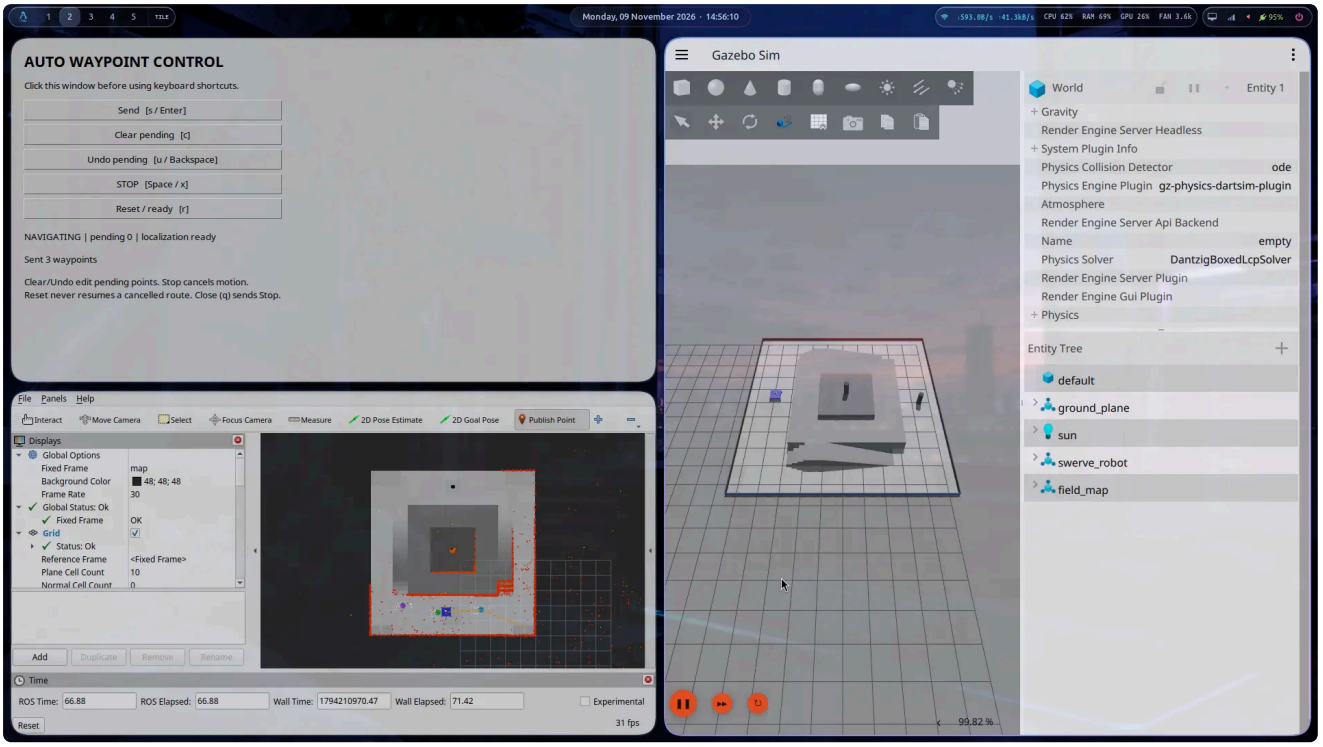
Limitations and Interpretation

- **No loop closure in this integration:** FAST-LIO2 supplies local LiDAR–inertial odometry and online mapping. Drift may accumulate. It does not automatically relocalize against the static field image or provide global position corrections.
- **Timing and initialization:** de-skewing requires usable per-point timing and consistent IMU timing. State initialization must use appropriate stationary measurements; this simulation gates out spawn free-fall/contact samples. Simulation timestamps are used throughout.
- **Geometry and noise:** sufficient geometric structure is needed to constrain pose. Our recorded flat-field failure and the scene-specific correction are documented separately below.
- **Control scope:** the robot follows operator-supplied waypoint segments. FAST-LIO itself does not add obstacle avoidance, global route planning or autonomous exploration.

Implementation

Workspace Layout

```
swerve_robot_sim/
├── Livox-SDK2/
├── ws_livox/ # supplies Livox ROS message
│   └── install/setup.zsh
├── livox_gz_sim/ # simulation plugin
│   └── install/setup.zsh
└── swerve_robot/
    ├── src/
    ├── patches/fast_lio_lyrical.patch
    └── scripts/
        ├── build_livox_sim.sh
        ├── run_livox_sim.sh
        └── record_livox_motion.sh
```



How to build

The commands below are for **Zsh**, the shell used in this workspace. Run them from:

```
cd /home/chheng-lydia/projects/dclab/swerve_robot_sim/swerve_robot
```

The dependency workspaces must already be built. The source snippets describe changes already present in this checkout; do not insert them a second time. For Bash terminals, use

the corresponding `setup.bash` files. The `.sh` helper scripts run Bash themselves and source their own environments.

Load ROS and Livox Dependencies

```
source /opt/ros/lyrical/setup.zsh
source ../ws_livox/install/setup.zsh
source ../livox_gz_sim/install/setup.zsh
```

> Sourcing makes installed ROS packages, generated messages, libraries and Gazebo plugins discoverable. Source all overlays in this same terminal before building.

Fetch the FAST-LIO source

If `src/fast_lio` does not exist, clone the source and initialize its submodule. Compilation is a later step.

```
git clone --branch ROS2 \
  https://github.com/hku-mars/FAST_LIO.git src/fast_lio

git -C src/fast_lio checkout cc3c9e8b1469b33b8072764d6db2348ed6dc597f

git -C src/fast_lio submodule update --init --recursive
```

Apply the code changes for `swerve_robot`

The following full files and edits match the current robot integration, including the keyboard-property fix, safe spawn defaults, startup gate and flat-field scan filter.

1. Mount the simulated Livox Mid-360

Open `src/swerve_robot/urdf/swerve_robot.urdf.xacro`.
Add these arguments alongside the existing `use_sim` argument:

```
<xacro:arg name="use_livox" default="false"/>
<xacro:arg name="livox_xfer_format" default="1"/>
<xacro:arg name="livox_downsample" default="1"/>
```

Inside the existing `<xacro:if value="$(arg use_sim)">` block, before `<ros2_control>`, insert:

```
<xacro:if value="$(arg use_livox)">
  <xacro:include filename="$(find livox_gz_sim)/urdf/mid360.xacro"/>
```

```
  <xacro:mid360 parent="base_link" name="livox"
                xfer_format="$(arg livox_xfer_format)"
                downsample="$(arg livox_downsample)">
    <origin xyz="0 0 ${base_z + base_height / 2 + 0.04}" rpy="0 0 0"/>
  </xacro:mid360>
</xacro:if>
```

This places the sensor above the chassis. Format `1` publishes Livox `CustomMsg`, including per-point timing for FAST-LIO. The imported macro also provides the IMU and sensor-frame transforms.

2. Enable Gazebo sensor rendering

Open `src/swerve_robot/worlds/empty.sdf`. Inside `<world>`, alongside the existing world plugins, add:

```
<plugin filename="gz-sim-sensors-system"
name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>
```

Keep the existing `gz-sim-imu-system` plugin. The Livox sensor uses GPU rendering, so server-only operation still needs working rendering support.

3. Configure FAST-LIO for the simulated sensor

Create `src/swerve_robot/config/fast_lio_mid360.yaml` with this content. The extrinsics describe the LiDAR origin in the sensor's IMU frame; robot mounting height is handled separately through URDF TF.

```
/**:
  ros__parameters:
    feature_extract_enable: false
    # Low ground returns are thinned by livox_startup_gate. Keep the
remaining
    # wall/platform returns instead of discarding another two thirds
here.
    point_filter_num: 1
    max_iteration: 3
    filter_size_surf: 0.3
    filter_size_map: 0.3
    # Keep the local-map cube wider than twice the FAST-LIO range margin.
    cube_side_length: 1000.0
```

```
    runtime_pos_log_enable: false
    common:
      lid_topic: /livox/lidar
      imu_topic: /livox/imu
      time_sync_en: false
      time_offset_lidar_to_imu: 0.0
    preprocess:
      lidar_type: 1
      scan_line: 4
      blind: 0.5
      timestamp_unit: 3
      scan_rate: 10
    mapping:
      acc_cov: 0.1
      gyr_cov: 0.1
      b_acc_cov: 0.0001
      b_gyr_cov: 0.0001
      fov_degree: 360.0
      det_range: 70.0
      extrinsic_est_en: false
      # LiDAR origin expressed in the IMU frame; matches
livox_gz_sim/mid360.xacro.
      extrinsic_T: [-0.011, -0.02329, 0.04412]
      extrinsic_R: [1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0]
    publish:
      path_en: true
      effect_map_en: false
      map_en: true
      scan_publish_en: true
      dense_publish_en: false
      scan_bodyframe_pub_en: true
    pcd_save:
      pcd_save_en: false
      interval: -1
```

The LiDAR and IMU use the same simulation clock, so leave software time synchronization disabled. Keep online extrinsic estimation disabled for this known simulated mounting geometry. The configuration enables live `/cloud_registered` and `/Laser_map`
publication and disables PCD saving.

4. Convert the FAST-LIO IMU pose into robot odometry

Create `src/swerve_robot/swerve_robot/fast_lio_odometry.py` with the complete implementation below.

The node reads `/Odometry` in FAST-LIO's `camera_init -> body` frames, obtains the sensor mounting transform from TF, removes the IMU lever arm, and normalizes the initial base pose to the odometry origin. It publishes `/odom`, `odom -> base_link`, and a fixed `odom -> camera_init` transform for the clouds. It also transforms pose covariance. Twist remains zero because this adapter does not estimate base velocity. It rejects invalid or non-increasing poses and does not substitute Gazebo ground truth when FAST-LIO stops.

```
"""Convert FAST-LIO's IMU pose into base_link odometry with a shared
cloud TF."""

from copy import deepcopy
import numpy as np
import rclpy
from rclpy.node import Node
from rclpy.time import Time
from geometry_msgs.msg import TransformStamped
from nav_msgs.msg import Odometry
from tf2_ros import Buffer, TransformListener,
StaticTransformBroadcaster, TransformBroadcaster, TransformException


def quaternion(q):
    value = np.array([q.x, q.y, q.z, q.w], dtype=float)
    norm = np.linalg.norm(value)
    if not np.isfinite(value).all() or norm < 1e-9:
        raise ValueError('Invalid orientation')
    return value / norm


def multiply(a, b):
    av, bv = a[:3], b[:3]
    return np.r_[a[3] * bv + b[3] * av + np.cross(av, bv),
                 a[3] * b[3] - np.dot(av, bv)]


def conjugate(q):
    return q * np.array([-1, -1, -1, 1])


def rotate(q, point):
```

```
    return multiply(multiply(q, np.r_[point, 0.0]), conjugate(q))[:3]


class PoseOrigin:
    """Remove IMU lever arm and normalize the initial base pose to odom
origin."""

    def __init__(self, imu_translation, imu_rotation):
        self.offset = np.asarray(imu_translation)
        self.mount = np.asarray(imu_rotation)
        self.origin = None

    def convert(self, position, orientation):
        base_q = multiply(orientation, conjugate(self.mount))
        base_p = np.asarray(position) - rotate(base_q, self.offset)
        if not np.isfinite(base_p).all():
            raise ValueError('Invalid position')
        if self.origin is None:
            self.origin = (base_p.copy(), base_q.copy())
        p0, q0 = self.origin
        return rotate(conjugate(q0), base_p - p0),
multiply(conjugate(q0), base_q)

    def world_transform(self):
        p0, q0 = self.origin
        return -rotate(conjugate(q0), p0), conjugate(q0)

    def covariance(self, covariance, imu_orientation):
        """Rotate pose uncertainty and account for the IMU lever arm."""
        q0 = self.origin[1]
        rotation = np.column_stack([rotate(conjugate(q0), axis) for axis
in np.eye(3)])
        arm = rotate(multiply(imu_orientation, conjugate(self.mount)),
self.offset)
        x, y, z = arm
        skew = np.array([[0, -z, y], [z, 0, -x], [-y, x, 0]])
        jacobian = np.zeros((6, 6))
        jacobian[:3, :3] = jacobian[3:, 3:] = rotation
        jacobian[:3, 3:] = rotation @ skew
        return (jacobian @ np.asarray(covariance).reshape(6, 6) @
jacobian.T).ravel().tolist()


def fill_pose(pose, position, orientation):
    pose.position.x, pose.position.y, pose.position.z = map(float,
```

```
position)
    pose.orientation.x, pose.orientation.y, pose.orientation.z,
pose.orientation.w = map(float, orientation)


class FastLioOdometry(Node):
    def __init__(self):
        super().__init__('fast_lio_odometry')
        self.buffer = Buffer()
        self.listener = TransformListener(self.buffer, self)
        self.static_tf = StaticTransformBroadcaster(self)
        self.tf = TransformBroadcaster(self)
        self.publisher = self.create_publisher(Odometry, '/odom', 10)
        self.create_subscription(Odometry, '/Odometry', self.receive, 10)
        self.origin = None
        self.last_stamp = None

    def receive(self, msg):
        if msg.header.frame_id != 'camera_init' or msg.child_frame_id !=
'body':
            self.get_logger().error('Expected FAST-LIO camera_init ->
body odometry', throttle_duration_sec=5)
            return
        stamp = Time.from_msg(msg.header.stamp).nanoseconds
        if self.last_stamp is not None and stamp <= self.last_stamp:
            return
        try:
            if self.origin is None:
                mount = self.buffer.lookup_transform('base_link',
'livox_imu_link', Time()).transform
                self.origin = PoseOrigin(
                    [mount.translation.x, mount.translation.y,
mount.translation.z],
                    quaternion(mount.rotation))
            raw = msg.pose.pose
            position, orientation = self.origin.convert(
                [raw.position.x, raw.position.y, raw.position.z],
quaternion(raw.orientation))
        except (TransformException, ValueError) as exc:
            self.get_logger().warning(f'Waiting for valid FAST-LIO pose
and sensor TF: {exc}', throttle_duration_sec=5)
            return
        if self.last_stamp is None:
            p, q = self.origin.world_transform()
            transform = self.make_transform(msg.header.stamp,
```

```
'camera_init', p, q)
            self.static_tf.sendTransform(transform)
            self.get_logger().info('FAST-LIO localized; initial base pose
is odom origin')
        self.last_stamp = stamp
        odom = deepcopy(msg)
        odom.header.frame_id = 'odom'
        odom.child_frame_id = 'base_link'
        fill_pose(odom.pose.pose, position, orientation)
        odom.pose.covariance =
self.origin.covariance(msg.pose.covariance, quaternion(raw.orientation))
        # FAST-LIO does not supply base_link twist. Do not pass off IMU
twist as base velocity.
        odom.twist = Odometry().twist
        self.publisher.publish(odom)
        self.tf.sendTransform(self.make_transform(msg.header.stamp,
'base_link', position, orientation))

    @staticmethod
    def make_transform(stamp, child, position, orientation):
        result = TransformStamped()
        result.header.stamp = stamp
        result.header.frame_id = 'odom'
        result.child_frame_id = child
        result.transform.translation.x, result.transform.translation.y,
result.transform.translation.z = map(float, position)
        result.transform.rotation.x, result.transform.rotation.y,
result.transform.rotation.z, result.transform.rotation.w = map(float,
orientation)
        return result


def main():
    rclpy.init()
    node = FastLioOdometry()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()
```

5. Adapt the current motor messages to Gazebo

Create `src/swerve_robot/swerve_robot/sim_motor_adapter.py` with the complete implementation below.

The existing IK produces `DCLabMotorCommand`, while the old simulation adapter expected another message format. Map command IDs 201–204 to FL, FR, RL, RR. M1 supplies wheel speed in rad/s; M2 supplies steering position in radians. Publish the resulting arrays to the two ros2_control command topics. Reject invalid commands and stop all four drive wheels if any module's command becomes older than 0.2 seconds.

```
"""Translate the current DCLab IK contract into Gazebo joint commands."""

import math
import time

import rclpy
from rclpy.node import Node
from custom_messages.msg import DCLabMotorCommand
from std_msgs.msg import Float64MultiArray


class MotorCommands:
    """Keep FL, FR, RL, RR ordered; a stale module stops the entire
drive."""

    def __init__(self, timeout=0.2):
        self.timeout = timeout
        self.speeds = [0.0] * 4
        self.angles = [0.0] * 4
        self.updated = [None] * 4

    def receive(self, msg, now):
        index = msg.can_id - 201
        if not 0 <= index < 4:
            return
        if (not msg.m1_enable or not msg.m2_enable
                or msg.mode1 != DCLabMotorCommand.MODE_SPEED
                or msg.mode2 != DCLabMotorCommand.MODE_POSITION
                or not math.isfinite(msg.goal1) or not
math.isfinite(msg.goal2)):
            self.updated[index] = None
            return
```

```
        self.speeds[index] = msg.goal1
        self.angles[index] = msg.goal2
        self.updated[index] = now

    def output(self, now):
        fresh = all(stamp is not None and now - stamp <= self.timeout for
stamp in self.updated)
        return (list(self.speeds) if fresh else [0.0] * 4),
list(self.angles)


class SimMotorAdapter(Node):
    def __init__(self):
        super().__init__('sim_motor_adapter')
        self.commands =
MotorCommands(float(self.declare_parameter('command_timeout',
0.2).value))
        self.drive_pub = self.create_publisher(Float64MultiArray,
'/wheel_velocity_controller/commands', 10)
        self.steer_pub = self.create_publisher(Float64MultiArray,
'/steering_position_controller/commands', 10)
        self.create_subscription(DCLabMotorCommand,
'/dclab/motor_command', self.receive, 20)
        self.create_timer(0.01, self.publish)

    def receive(self, msg):
        self.commands.receive(msg, time.monotonic())

    def publish(self):
        speeds, angles = self.commands.output(time.monotonic())
        self.drive_pub.publish(Float64MultiArray(data=speeds))
        self.steer_pub.publish(Float64MultiArray(data=angles))


def main():
    rclpy.init()
    node = SimMotorAdapter()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if rclpy.ok():
            node.drive_pub.publish(Float64MultiArray(data=[0.0] * 4))
        node.destroy_node()
```

```
        if rclpy.ok():
            rclpy.shutdown()
```

6. Add automatic waypoint control to the main controller

Open `src/swerve_robot/swerve_robot/main.py`. Keep the existing waypoint follower and its stale-odometry/path-error checks.

6.1 Restore the missing joint constant

Add the previously undefined constant beside `SUCTION_VESC_ID`:

```
JOINT_VESC_ID = 1
SUCTION_VESC_ID = 2
```

6.2 Declare the automatic simulation mode

In `Main.__init__`, immediately after the parent constructor, add:

```
self.simulation_auto = self.declare_parameter('simulation_auto',
False).value
```

After `self.init_publisher()` and `self.init_subscriber()`, add:

```
if self.simulation_auto:
    self.set_mobility(MobilityState.SEMI_AUTO)
    self.get_logger().info('Simulation auto ready: queue points, then
Send')
```

6.3 Disable gamepad subscriptions only in this mode

In `init_subscriber`, replace the two gamepad subscription conditions with:

```
if not self.simulation_auto and self.gamepad_source in ('auto',
'gamepad2026'):
```

and:

```
if not self.simulation_auto and self.gamepad_source in ('auto',
'legacy'):
```

Keep the existing subscription bodies under those conditions.

6.4 Register the operator services

Keep the existing `/undo_waypoint` service. After that registration in `init_subscriber`, insert:

```
if self.simulation_auto:
    self.operator_services = [
        self.create_service(Trigger, name, callback)
        for name, callback in (
            ('/send_waypoints', self.send_waypoints_cb),
            ('/clear_waypoints', self.clear_waypoints_cb),
            ('/stop_route', self.stop_route_cb),
            ('/reset_route', self.reset_route_cb),
            ('/route_status', self.route_status_cb),
        )
    ]
```

`Trigger` is already imported in the existing controller. Add the following methods inside the `Main` class, after `undo_waypoint_cb`:

```
    def send_waypoints_cb(self, request, response):
        nav = self.semi_auto
        if self.estop_active or nav.state == SemiAutoState.FAULT:
            response.message = 'Stopped or faulted: Reset before sending
a new route'
        elif nav.state != SemiAutoState.WAITING or nav.goal_pending:
            response.message = 'A route is already running'
        elif not self.pending_waypoints:
            response.message = 'No pending waypoints; add points in RViz'
        elif (nav.pose is None or not all(math.isfinite(v) for v in
nav.pose)
              or time.perf_counter() - nav.pose_time > nav.odom_timeout):
            response.message = 'Waiting for fresh localization'
        else:
            nav.set_waypoint(self.pending_waypoints)
            self.route_waypoints = list(self.pending_waypoints)
            self.pending_waypoints.clear()
            response.success = True
            response.message = f'Sent {len(self.route_waypoints)}
waypoints'
            self._refresh_waypoint_markers()
        return response

    def clear_waypoints_cb(self, request, response):
```

```
        self.pending_waypoints.clear()
        self._refresh_waypoint_markers()
        response.success = True
        response.message = 'Pending points cleared; active route is
unchanged'
        return response

    def stop_route_cb(self, request, response):
        self.estop_active = True
        self.semi_auto.enter()
        self.limiter.reset()
        self._publish_cmd_vel(*ZERO_CMD)
        self._refresh_waypoint_markers()
        response.success = True
        response.message = 'Stopped; active route cancelled. Reset to
accept a new route'
        return response

    def reset_route_cb(self, request, response):
        self.estop_active = False
        self.limiter.reset()
        self._publish_cmd_vel(*ZERO_CMD)
        self.set_mobility(MobilityState.SEMI_AUTO)
        self._refresh_waypoint_markers()
        response.success = True
        response.message = 'Auto ready; queue/send a new route (no
automatic resume)'
        return response

    def route_status_cb(self, request, response):
        nav = self.semi_auto
        fresh = nav.pose is not None and time.perf_counter() -
nav.pose_time <= nav.odom_timeout
        response.success = fresh and not self.estop_active and nav.state
!= SemiAutoState.FAULT
        state = 'STOPPED' if self.estop_active else nav.state.name
        response.message = (f'{state} | pending
{len(self.pending_waypoints)} | '
                            f'localization {"ready" if fresh else
"waiting"}'
                            + (f' | {nav.fault_reason}' if
nav.fault_reason else ''))
        return response
```

Send requires a pending route and fresh localization. Stop cancels the active route and stays latched. Reset returns to ready without resuming the cancelled route. Clear and Undo affect pending points only.

6.5 Keep the gamepad watchdog for gamepad mode

In `timer_cb`, replace:

```
expired = self._pad_command_expired()
```

with:

```
expired = not self.simulation_auto and self._pad_command_expired()
```

The existing odometry timeout and path-error fault remain active in automatic mode.

6.6 Skip physical mechanism commands in automatic simulation

In `timer_cb`, wrap the existing mechanism calls as follows:

```
if not self.simulation_auto:
    self.scroll.update(self.a_pressed, self.y_pressed, self.rb_pressed)
    self.scroll_mechanism_control()
    self.controller_board_control()
    self.suction_control(expired)
```

6.7 Reject invalid waypoints

At the beginning of `_add_waypoint`, before converting the waypoint frame, add:

```
if not all(math.isfinite(value) for value in (x, y)) or (yaw is not None
and not math.isfinite(yaw)):
    self.get_logger().warning('Ignoring non-finite waypoint')
    return
```

6.8 Publish zero velocity on shutdown

In the module's `main()` cleanup, replace the controller-board-only shutdown block with:

```
if node is not None:
    if rclpy.ok():
        node.estop_active = True
```

```
        node._publish_cmd_vel(*ZERO_CMD)
        if not node.simulation_auto:
            node.controller_board_control()
    node.destroy_node()
```

Keep the existing `rclpy.shutdown()` handling afterward.

7. Create the keyboard control panel

Create `src/swerve_robot/swerve_robot/keyboard_control.py` using this complete implementation. ROS launch does not forward ordinary terminal input to this node, so use a focusable Tkinter window with buttons and shortcuts. The panel calls the controller's Trigger services and displays route/localization status. Closing the panel requests Stop.

```
"""A focusable keyboard/button panel; ROS launch does not forward
terminal stdin."""

import tkinter as tk
from tkinter import ttk

import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger


COMMANDS = (
    ('Send', 's / Enter', '/send_waypoints'),
    ('Clear pending', 'c', '/clear_waypoints'),
    ('Undo pending', 'u / Backspace', '/undo_waypoint'),
    ('STOP', 'Space / x', '/stop_route'),
    ('Reset / ready', 'r', '/reset_route'),
)
KEYS = {'s': 0, 'Return': 0, 'c': 1, 'u': 2, 'BackSpace': 2,
        'space': 3, 'x': 3, 'r': 4}


class KeyboardControl(Node):
    def __init__(self):
        super().__init__('simulation_keyboard')
        self.window = tk.Tk()
        self.window.title('Swerve · FAST-LIO2 · Auto control')
        self.command_clients = {topic: self.create_client(Trigger, topic)
                                for _, _, topic in COMMANDS}
```

```
        self.status_client = self.create_client(Trigger, '/route_status')
        self.pending = {}
        self.closing = False
        self.status = tk.StringVar(value='Waiting for controller and
localization…')
        self.feedback = tk.StringVar(value='Add points with RViz Publish
Point or 2D Goal Pose.')
        frame = ttk.Frame(self.window, padding=18)
        frame.grid()
        ttk.Label(frame, text='AUTO WAYPOINT CONTROL', font=('', 16,
'bold')).grid(sticky='w')
        ttk.Label(frame, text='Click this window before using keyboard
shortcuts.').grid(sticky='w', pady=8)
        for row, (label, key, topic) in enumerate(COMMANDS, 2):
            ttk.Button(frame, text=f'{label}   [{key}]',
                       command=lambda t=topic:
self.send(t)).grid(row=row, sticky='ew', pady=3)
        ttk.Label(frame, textvariable=self.status,
wraplength=500).grid(sticky='w', pady=12)
        ttk.Label(frame, textvariable=self.feedback,
wraplength=500).grid(sticky='w')
        ttk.Label(frame, text='Clear/Undo edit pending points. Stop
cancels motion.\n'
                  'Reset never resumes a cancelled route. Close (q) sends
Stop.').grid(sticky='w', pady=12)
        self.window.bind('<KeyPress>', self.key)
        self.window.protocol('WM_DELETE_WINDOW', self.close)
        self.create_timer(0.5, self.poll_status)

    def key(self, event):
        key = event.keysym
        if key in KEYS:
            self.send(COMMANDS[KEYS[key]][2])
            return 'break'
        if key == 'q':
            self.close()
            return 'break'

    def send(self, topic):
        client = self.command_clients[topic]
        if topic in self.pending:
            return
        if not client.service_is_ready():
            self.feedback.set('Controller is not ready yet')
            return
```

```
        future = client.call_async(Trigger.Request())
        self.pending[topic] = future

        def done(result):
            self.pending.pop(topic, None)
            try:
                self.feedback.set(result.result().message)
            except Exception as exc:
                self.feedback.set(str(exc))
            if self.closing and topic == '/stop_route':
                self.window.quit()
        future.add_done_callback(done)

    def poll_status(self):
        if '/route_status' in self.pending or not
self.status_client.service_is_ready():
            return
        future = self.status_client.call_async(Trigger.Request())
        self.pending['/route_status'] = future

        def done(result):
            self.pending.pop('/route_status', None)
            try:
                self.status.set(result.result().message)
            except Exception:
                self.status.set('Controller unavailable')
        future.add_done_callback(done)

    def close(self):
        self.closing = True
        self.send('/stop_route')
        # Allow the stop response to arrive before closing; avoid hanging
on a dead controller.
        self.window.after(1500, self.window.quit)

    def spin_once(self):
        if not rclpy.ok():
            self.window.quit()
            return
        rclpy.spin_once(self, timeout_sec=0)
        self.window.after(10, self.spin_once)


def main():
    rclpy.init()
```

```
    node = None
    try:
        node = KeyboardControl()
        node.spin_once()
        node.window.mainloop()
    except (KeyboardInterrupt, tk.TclError) as exc:
        if node is None:
            print(f'Cannot open keyboard panel: {exc}. Use
keyboard:=false for headless launch.')
    finally:
        if node is not None:
            node.window.destroy()
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()
```

8. Update the existing simulation launch

Open `src/swerve_robot/launch/swerve_robot.launch.py`.

8.1 Import the condition helper

Add `PythonExpression` to the existing imports from `launch.substitutions`:

```
from launch.substitutions import (
    Command,
    PythonExpression,
    FindExecutable,
    LaunchConfiguration,
    PathJoinSubstitution,
)
```

8.2 Declare the new launch arguments and sensor resource path

Insert these actions at the start of the returned `LaunchDescription` list:

```
DeclareLaunchArgument('use_livox', default_value='false'),
DeclareLaunchArgument('livox_xfer_format', default_value='1', choices=
['0', '1']),
DeclareLaunchArgument('livox_downsample', default_value='1'),
DeclareLaunchArgument('simulation_auto', default_value='false'),
DeclareLaunchArgument('rviz_config', default_value=PathJoinSubstitution([
    FindPackageShare('swerve_robot'), 'maps', 'default.rviz'])),
AppendEnvironmentVariable(
```

```
    'GZ_SIM_RESOURCE_PATH',
    PathJoinSubstitution([FindPackageShare('livox_gz_sim'), '..']),
    condition=IfCondition(LaunchConfiguration('use_livox')),
),
```

These are entries in the existing list, not a separate launch description.

8.3 Make the RViz configuration selectable

Replace the fixed `rviz_config` assignment inside `generate_launch_description` with:

```
rviz_config = LaunchConfiguration('rviz_config')
```

8.4 Forward the sensor arguments to Xacro

At the end of the existing `robot_description_content = Command([...])` argument list, replace the `use_sim` argument segment with:

```
'use_sim:=', use_gazebo,
' use_livox:=', LaunchConfiguration('use_livox'),
' livox_xfer_format:=', LaunchConfiguration('livox_xfer_format'),
' livox_downsample:=', LaunchConfiguration('livox_downsample')
```

Keep the earlier executable, Xacro filename, and separating space in the list.

8.5 Select the new motor adapter

Replace the existing `can_driver` node definition with:

```
can_driver = Node(
    package='swerve_robot',
    executable='sim_motor_adapter',
    output='screen',
    parameters=[{
        'command_timeout': 0.2,
        'use_sim_time': use_sim_time,
    }],
    condition=IfCondition(use_gazebo),
)
```

The variable name remains `can_driver` so its existing entry in the launch list remains valid. The node now runs the simulation-only Python adapter.

8.6 Allow FAST-LIO as the odometry source

In the existing `odometry_source` launch argument, expand the choices:

```
choices=['ground_truth', 'wheel', 'fast_lio'],
```

Replace the `condition` on the existing `sim_odometry` node with:

```
condition=IfCondition(PythonExpression([
    "'", use_gazebo, "' == 'true' and '",
    LaunchConfiguration('odometry_source'), "' != 'fast_lio'"
])),
```

This prevents `sim_odometry` and the FAST-LIO adapter from both publishing `/odom` and `odom -> base_link`. Selecting `fast_lio` in the base launch only turns off ordinary simulation odometry; the wrapper launch in the next step starts FAST-LIO and its adapter.

8.7 Forward automatic mode to the controller

Add this entry to the existing `main_node` parameters dictionary:

```
'simulation_auto': ParameterValue(
    LaunchConfiguration('simulation_auto'), value_type=bool),
```

Keep `'use_odom_yaw': True` so the controller uses the incoming odometry heading.

8.8 Wait for the simulated IMU and balance flat-field scans

Create `src/swerve_robot/swerve_robot/livox_startup_gate.py` with the following content.
The robot is spawned above the floor. Initial free-fall and contact samples must not enter FAST-LIO's gravity/acceleration-scale initialization. This gate waits for one continuous second of IMU samples with acceleration magnitude 0.9–1.1 g and angular speed below 0.05 rad/s, using sensor timestamps. Gaps over 0.05 s restart the window. The Livox simulator publishes acceleration in g.

It forwards subsequent IMU samples and scans whose start timestamp is after the gate opens to `/fast_lio/imu` and `/fast_lio/lidar`. IMU values and timestamps are preserved. The retained LiDAR points keep their original per-point timestamps and order; the point count is updated. After opening, ordinary robot motion does not

close the gate. A backward time jump blocks forwarding until the launch is restarted. This does not add map relocalization or detect later estimator drift.

For this level-mounted sensor on the flat field, the wrapper sets `ground_keep_every=200`: keep every higher point and one in 200 low returns (`z <= -0.15 m` in the LiDAR frame). The original raw topic stays available. This prevents noisy, dense floor returns from dominating scan matching; it is not a ground classifier for slopes or a tilted robot. Set `ground_keep_every:=1` to disable thinning for other scenes, then retune and validate localization. FAST-LIO keeps the remaining points (`point_filter_num=1`) and uses 0.3 m surface/map voxels. See recorded validation.

```
"""Settle the simulated IMU and balance flat-field scans before FAST-
LIO."""

from copy import copy
import math

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu
from livox_ros_driver2.msg import CustomMsg


class SettlingGate:
    """Open once after steady Livox IMU data (acceleration is expressed
in g)."""

    def __init__(self, duration=1.0):
        self.duration = duration
        self.since = None
        self.last_stamp = None
        self.ready_stamp = None
        self.reset_detected = False

    def receive(self, stamp, acceleration, angular_velocity):
        if self.last_stamp is not None and stamp < self.last_stamp:
            # FAST-LIO and its odometry origin also need restarting after
a reset.
            self.reset_detected = True
        if self.reset_detected:
            return False
        if self.last_stamp is not None and stamp == self.last_stamp:
```

```
            return False
        gap = self.last_stamp is not None and stamp - self.last_stamp >
0.05
        self.last_stamp = stamp
        if self.ready_stamp is not None:
            return True  # Normal driving must not close a startup-only
gate.
        acc = math.hypot(*acceleration)
        gyro = math.hypot(*angular_velocity)
        steady = math.isfinite(acc) and math.isfinite(gyro) and 0.9 <=
acc <= 1.1 and gyro < 0.05
        if not steady:
            self.since = None
            return False
        if self.since is None or gap:
            self.since = stamp
        if stamp - self.since >= self.duration:
            self.ready_stamp = stamp
            return True
        return False

    def accepts_scan(self, stamp):
        return (not self.reset_detected and self.ready_stamp is not None
                and stamp >= self.ready_stamp)


def seconds(stamp):
    return stamp.sec + stamp.nanosec * 1e-9


def balance_cloud(msg, ground_z_max, ground_keep_every):
    """Thin the low returns of this level-mounted, flat-field simulation.

    Keep all higher returns and one in N low returns. Preserve scan order
and
    per-point timestamps for de-skewing. N=1 disables thinning. This is a
sensor
    height cutoff, not a terrain/ground classifier for slopes or a tilted
robot.
    """
    points = []
    low_count = 0
    for point in msg.points:
        if not all(math.isfinite(v) for v in (point.x, point.y,
point.z)):
```

```
            continue
        if point.z <= ground_z_max:
            keep = low_count % ground_keep_every == 0
            low_count += 1
            if not keep:
                continue
        points.append(point)
    result = copy(msg)
    result.points = points
    result.point_num = len(points)
    return result


class LivoxStartupGate(Node):
    def __init__(self):
        super().__init__('livox_startup_gate')
        self.gate = SettlingGate()
        self.ground_z_max = float(self.declare_parameter('ground_z_max',
-0.15).value)
        self.ground_keep_every =
int(self.declare_parameter('ground_keep_every', 1).value)
        if not math.isfinite(self.ground_z_max) or self.ground_keep_every
< 1:
            raise ValueError('ground_z_max must be finite and
ground_keep_every must be >= 1')
        self.imu_pub = self.create_publisher(Imu, '/fast_lio/imu', 200)
        self.lidar_pub = self.create_publisher(CustomMsg,
'/fast_lio/lidar', 10)
        self.create_subscription(Imu, '/livox/imu', self.imu, 200)
        self.create_subscription(CustomMsg, '/livox/lidar', self.lidar,
10)
        self.get_logger().info('Waiting for 1 s of steady IMU data before
FAST-LIO initialization')
        self.get_logger().info(
            f'Flat-field scan filter: keep 1/{self.ground_keep_every}
returns '
            f'at LiDAR z <= {self.ground_z_max} m; keep all higher
returns')

    def imu(self, msg):
        was_ready = self.gate.ready_stamp is not None
        a, w = msg.linear_acceleration, msg.angular_velocity
        if self.gate.receive(seconds(msg.header.stamp), (a.x, a.y, a.z),
(w.x, w.y, w.z)):
            if not was_ready:
```

```
                self.get_logger().info('IMU settled; forwarding new scans
and IMU samples to FAST-LIO')
            self.imu_pub.publish(msg)
        if self.gate.reset_detected:
            self.get_logger().error('Simulation time reset: restart the
Livox launch', throttle_duration_sec=5)

    def lidar(self, msg):
        if self.gate.accepts_scan(seconds(msg.header.stamp)):
            self.lidar_pub.publish(balance_cloud(msg, self.ground_z_max,
self.ground_keep_every))


def main():
    rclpy.init()
    node = LivoxStartupGate()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()
```

9. Add the combined Livox launch

Create `src/swerve_robot/launch/livox_sim.launch.py` with the following content. This wrapper enables Livox CustomMsg output, FAST-LIO odometry, automatic waypoint mode, the dedicated RViz configuration, and the keyboard panel. Its default map poses are (-0.5, 0.5) for blue and (-10.5, 0.5) for red, facing west. These leave room for the wheels and steering modules to clear the 3 cm field walls; the older hardware corner presets intersect the simulated wall geometry.

```
"""One-command Mid-360 simulation, FAST-LIO2 mapping and automatic
waypoints."""

import math

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument,
IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import
```

```
PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration,
PathJoinSubstitution, PythonExpression
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare


def generate_launch_description():
    package = FindPackageShare('swerve_robot')
    return LaunchDescription([
        DeclareLaunchArgument('keyboard', default_value='true'),
        DeclareLaunchArgument('use_rviz', default_value='true'),
        DeclareLaunchArgument('use_nvidia', default_value='false'),
        DeclareLaunchArgument('gz_extra_args', default_value=''),
        DeclareLaunchArgument('start_side', default_value='blue',
choices=['blue', 'red']),
        # Hardware corner presets intersect the simulated 3 cm perimeter
walls.
        # Leave room for the full steering/wheel envelope on either team
side.
        DeclareLaunchArgument('start_x', default_value=PythonExpression([
            "-0.5 if '", LaunchConfiguration('start_side'), "' == 'blue'
else -10.5",
        ]), description='Simulation start X in map, with perimeter
clearance'),
        DeclareLaunchArgument('start_y', default_value='0.5',
                              description='Simulation start Y in map,
clear of the south wall'),
        DeclareLaunchArgument('start_yaw', default_value=str(math.pi)),
        DeclareLaunchArgument('v_max', default_value='0.3'),
        DeclareLaunchArgument('livox_downsample', default_value='1'),
        DeclareLaunchArgument('ground_keep_every', default_value='200',
                              description='Keep one in N low returns for
flat-field LIO; 1 disables'),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(PathJoinSubstitution([package,
'launch', 'swerve_robot.launch.py'])),
            launch_arguments={
                'use_livox': 'true', 'livox_xfer_format': '1',
                'simulation_auto': 'true', 'odometry_source': 'fast_lio',
                'use_sim_time': 'true', 'use_gazebo': 'true',
                'use_rviz': LaunchConfiguration('use_rviz'),
                'use_nvidia': LaunchConfiguration('use_nvidia'),
                'gz_extra_args': LaunchConfiguration('gz_extra_args'),
                'start_side': LaunchConfiguration('start_side'),
```

```
                'start_x': LaunchConfiguration('start_x'),
                'start_y': LaunchConfiguration('start_y'),
                'start_yaw': LaunchConfiguration('start_yaw'),
                'v_max': LaunchConfiguration('v_max'),
                'livox_downsample':
LaunchConfiguration('livox_downsample'),
                'rviz_config': PathJoinSubstitution([package, 'maps',
'livox.rviz']),
            }.items()),
        Node(package='swerve_robot', executable='livox_startup_gate',
output='screen',
             parameters=[{'use_sim_time': True, 'ground_z_max': -0.15,
                          'ground_keep_every':
LaunchConfiguration('ground_keep_every')}]),
        Node(package='fast_lio', executable='fastlio_mapping',
output='screen',
             parameters=[PathJoinSubstitution([package, 'config',
'fast_lio_mid360.yaml']),
                         {'use_sim_time': True}],
             remappings=[('/livox/lidar', '/fast_lio/lidar'),
                         ('/livox/imu', '/fast_lio/imu')]),
        Node(package='swerve_robot', executable='fast_lio_odometry',
output='screen',
             parameters=[{'use_sim_time': True}]),
        # Wall-clock event loop stays responsive even when Gazebo is
paused.
        Node(package='swerve_robot', executable='keyboard_control',
output='screen',
             condition=IfCondition(LaunchConfiguration('keyboard'))),
    ])
```

10. Add the RViz displays

Copy `src/swerve_robot/maps/default.rviz` to `src/swerve_robot/maps/livox.rviz`. Retain the existing field, robot, and waypoint displays. Append these entries to the `Visualization Manager` -> `Displays` list, matching the indentation of the existing entries:

```
- Class: rviz_default_plugins/PointCloud2
  Name: Livox registered scan
  Enabled: true
  Topic:
    Value: /cloud_registered
    Depth: 5
```

```
    Reliability Policy: Reliable
    Durability Policy: Volatile
    History Policy: Keep Last
  Size (Pixels): 2
  Size (m): 0.025
  Style: Points
  Decay Time: 0.3
  Position Transformer: XYZ
  Color Transformer: AxisColor
  Axis: Z
  Alpha: 0.8
  Value: true
- Class: rviz_default_plugins/PointCloud2
  Name: FAST-LIO map
  Enabled: true
  Topic:
    Value: /Laser_map
    Depth: 5
    Reliability Policy: Reliable
    Durability Policy: Volatile
    History Policy: Keep Last
  Size (Pixels): 2
  Size (m): 0.025
  Style: Points
  Decay Time: 0.0
  Position Transformer: XYZ
  Color Transformer: AxisColor
  Axis: Z
  Alpha: 0.8
  Value: true
- Class: rviz_default_plugins/Path
  Name: FAST-LIO trajectory
  Enabled: true
  Topic:
    Value: /path
    Depth: 5
    Reliability Policy: Reliable
    Durability Policy: Volatile
    History Policy: Keep Last
  Color: 255; 170; 0
  Line Style: Lines
  Line Width: 0.03
  Value: true
```

The field image remains `/map`. The reconstructed 3D map is `/Laser_map`; these are separate data products. The pose adapter connects the FAST-LIO cloud frame to the existing field/robot TF tree.

11. Register executables and dependencies

In `src/swerve_robot/setup.py`, add these entries to the existing `entry_points['console_scripts']` list:

```
'sim_motor_adapter = swerve_robot.sim_motor_adapter:main',
'keyboard_control = swerve_robot.keyboard_control:main',
'fast_lio_odometry = swerve_robot.fast_lio_odometry:main',
'livox_startup_gate = swerve_robot.livox_startup_gate:main',
```

The package's existing data-file rules install launch files, configurations, URDFs, worlds, and RViz presets, so no additional installation rule is needed.

In `src/swerve_robot/package.xml`, add:

```
<exec_depend>python3-tk</exec_depend>
<exec_depend>livox_gz_sim</exec_depend>
<exec_depend>livox_ros_driver2</exec_depend>
<exec_depend>fast_lio</exec_depend>
```

Apply the ROS Lyrical fix

In `src/fast_lio/CMakeLists.txt`:

1. **Change minimum CMake version**

```
cmake_minimum_required(VERSION 3.16)
```

2. **Remove these lines**

```
ADD_COMPILE_OPTIONS(-std=c++14)
ADD_COMPILE_OPTIONS(-std=c++14)
set(CMAKE_CXX_FLAGS "-std=c++14 -O3")
```

3. **Update C++ setting**

```
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -pthread -fexceptions")
```

4. Replace `find_package(PythonLibs REQUIRED)` with

```
find_package(Python3 REQUIRED COMPONENTS Interpreter Development)
```

5. Add this

```
find_package(tf2_ros REQUIRED)
```

6. Update Python linking and includes

```
target_link_libraries(fastlio_mapping
  ${PCL_LIBRARIES}
  Python3::Python
  Eigen3::Eigen
)

target_include_directories(fastlio_mapping PRIVATE
  ${Python3_INCLUDE_DIRS}
)
```

7. Replace `ament_target_dependencies(fastlio_mapping ${dependencies})` with this

```
target_link_libraries(fastlio_mapping
  rclcpp::rclcpp
  tf2_ros::tf2_ros
  pcl_conversions::pcl_conversions
  ${geometry_msgs_TARGETS}
  ${nav_msgs_TARGETS}
  ${sensor_msgs_TARGETS}
  ${std_msgs_TARGETS}
  ${std_srvs_TARGETS}
  ${visualization_msgs_TARGETS}
  ${livox_ros_driver2_TARGETS}
)
```

These fixes update:

- CMake dependency linking to use exported targets

- Python discovery to use `Python3` and `Python3::Python`
- The project's requested C++ standard to C++17, removing conflicting older flags; ROS dependencies can require a newer standard (this build uses C++20)
- The TF broadcaster include to `tf2_ros/transform_broadcaster.hpp`
- The explicit `tf2_ros` dependency

Update TF Broadcaster include

In `src/fast_lio/src/laserMapping.cpp`, change `#include <tf2_ros/transform_broadcaster.h>` with `#include <tf2_ros/transform_broadcaster.hpp>`

Declare dependency

In `src/fast_lio/package.xml`, add this dependency

```
<depend>tf2_ros</depend>
```

These changes provide compatibility with this ROS 2 Lyrical installation. FAST-LIO's mapping and estimation algorithms remain unchanged.

Compile

Run from the `swerve_robot` workspace directory after sourcing ROS and the Livox dependencies:

```
export CMAKE_BUILD_PARALLEL_LEVEL=2

colcon build \
  --packages-select custom_messages fast_lio swerve_robot \
  --parallel-workers 2 \
  --cmake-clean-cache \
  --cmake-args \
    -DCMAKE_BUILD_TYPE=Release \
    -DPython3_EXECUTABLE=/usr/bin/python3
```

- `custom_messages`: builds the robot's shared ROS interfaces.
- `fast_lio`: builds FAST-LIO2.
- `swerve_robot`: installs the robot controllers, simulation adapters, configuration, and launch files.
- `--cmake-clean-cache`: removes cached paths from the previous workspace location.
- `Release`: enables optimized compilation.

- `/usr/bin/python3`: selects the Python interpreter used by this ROS installation.

Source the completed workspace

```
source install/setup.zsh
```

Rebuild and source the workspace again after modifying source or configuration.

Launch the simulation

From the sourced `swerve_robot` workspace:

```
ros2 launch swerve_robot livox_sim.launch.py
```

This starts Gazebo, the simulated Livox Mid-360, the startup/scan-filter gate, FAST-LIO2, robot odometry, waypoint control, RViz and the keyboard panel. Wait for `IMU settled` and fresh localization before sending a route. The gate forwards to `/fast_lio/lidar` and `/fast_lio/imu`; the original `/livox/*` sensor topics stay available.

For NVIDIA PRIME offload:

```
ros2 launch swerve_robot livox_sim.launch.py use_nvidia:=true
```

To select the red starting zone and reduce speed:

```
ros2 launch swerve_robot livox_sim.launch.py start_side:=red v_max:=0.2
```

Verify the sensor and localization topics

Open another Zsh terminal and source the environments:

```
cd /home/chheng-lydia/projects/dclab/swerve_robot_sim/swerve_robot

source /opt/ros/lyrical/setup.zsh
source ../ws_livox/install/setup.zsh
source ../livox_gz_sim/install/setup.zsh
source install/setup.zsh
```

Run these checks separately. Stop each continuous command with Ctrl+C:

```
ros2 topic hz /livox/lidar
ros2 topic hz /livox/imu
```

```
ros2 topic echo /odom --once
ros2 run tf2_ros tf2_echo odom base_link
```

Check the waypoint controller:

```
ros2 service call /route_status std_srvs/srv/Trigger '{}'
```

Wait for `IMU settled` and `localization ready` before sending a route. Fresh odometry alone does not establish accuracy: check that the stationary robot stays near its start and that RViz follows Gazebo during a short test route.

Execute a waypoint route

1. Keep the robot stationary while FAST-LIO initializes.
2. In RViz, select **Publish Point** to queue positions.
3. Use **2D Goal Pose** if a waypoint also requires a final heading.
4. Review the route and choose unobstructed segments.
5. Focus the keyboard panel and press **s** or **Enter**.

Key	Action
s / Enter	Send pending waypoints
c	Clear pending waypoints
u / Backspace	Undo the last pending waypoint
Space / x	Stop and cancel the active route
r	Reset the stop/fault state
q	Request Stop and close the keyboard panel

Clear and Undo affect pending points only. Reset does not resume a cancelled route. Automatic mode waits for a route to be sent; it does not explore by itself.

Shut down

Press **Ctrl+C in the launch terminal** to stop the complete simulation.

Restart the full launch after resetting Gazebo's world or simulation time.

> This simulation is tested on ROS 2 lyrical with Gazebo Jetty on Ubuntu 26.04. If you're using other versions, you may need to adjust the launch arguments, dependencies, or FAST-LIO compilation flags.